

Rapport du Mini-Projet

Beware of poisonous mushrooms

Contexte :

Nous avons étudié un ensemble de données sur les champignons afin de prédire s'ils sont comestibles ou non à partir de leurs caractéristiques.

Objectif :

L'objectif de ce projet est de construire des modèles de machine learning capables de prédire la classe d'un champignon (comestible ou vénéneux), en se basant uniquement sur ses caractéristiques. Pour cela, nous avons utilisé deux techniques :

- **K-Nearest Neighbors (KNN)** : un algorithme qui classe un élément en fonction des éléments similaires autour de lui.
- **Arbre de décision** : un modèle qui prend des décisions étape par étape en fonction des valeurs des caractéristiques.

Étape 1 : Chargement et préparation des données :

Dans cette étape, nous avons chargé les données à partir d'un fichier CSV. Chaque ligne représente un champignon, et chaque colonne contient des caractéristiques spécifiques (par exemple, la couleur du chapeau, l'odeur, la forme du pied). Les colonnes ont été nommées pour faciliter leur interprétation, avec "class" indiquant si le champignon est comestible ou vénéneux, et les autres colonnes décrivant des caractéristiques physiques du champignon.

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...
0	p	x	s	n	t	p	f	c	n	k	...
1	e	x	s	y	t	a	f	c	b	k	...
2	e	b	s	w	t	l	f	c	b	n	...
3	p	x	y	w	t	p	f	c	n	n	...
4	e	x	s	g	f	n	f	w	b	k	...

5 rows × 23 columns

Mariam Boughizene

Mohsen Allani

Étape 2 : Traitement des valeurs manquantes :

Nous avons détecté des valeurs manquantes dans la colonne "stalk-root". Ces valeurs ont été remplacées par la valeur la plus fréquente (mode) de cette colonne. Après ce traitement, il n'y a plus de valeurs manquantes dans le jeu de données.

Étape 3 : Visualisation de la distribution des données :

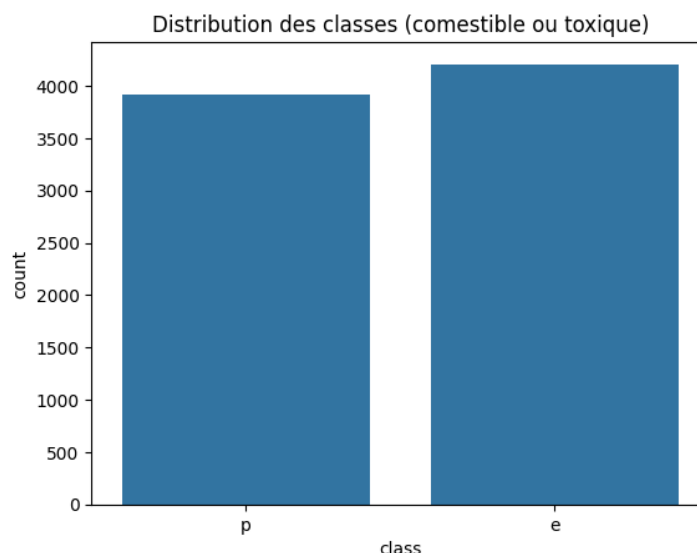
Les visualisations, comme **les graphes de comptage** et **les graphes de type mosaïque**, sont essentielles dans l'analyse de données pour plusieurs raisons

1. Compréhension de la Distribution des Données
2. Détection de Patterns et de Relations
3. Préparation des Données pour les Modèles
4. Évaluation des Hypothèses Initiales

Nous avons généré des graphiques de comptage pour chaque caractéristique afin d'explorer la distribution des valeurs dans chaque colonne. Ensuite, un graphique a été créé pour la colonne "**class**" afin de visualiser la répartition des champignons comestibles et toxiques.

Dans ce graphique, nous pouvons observer le nombre de champignons comestibles et toxiques.

- **e : comestible**
- **p : toxique**

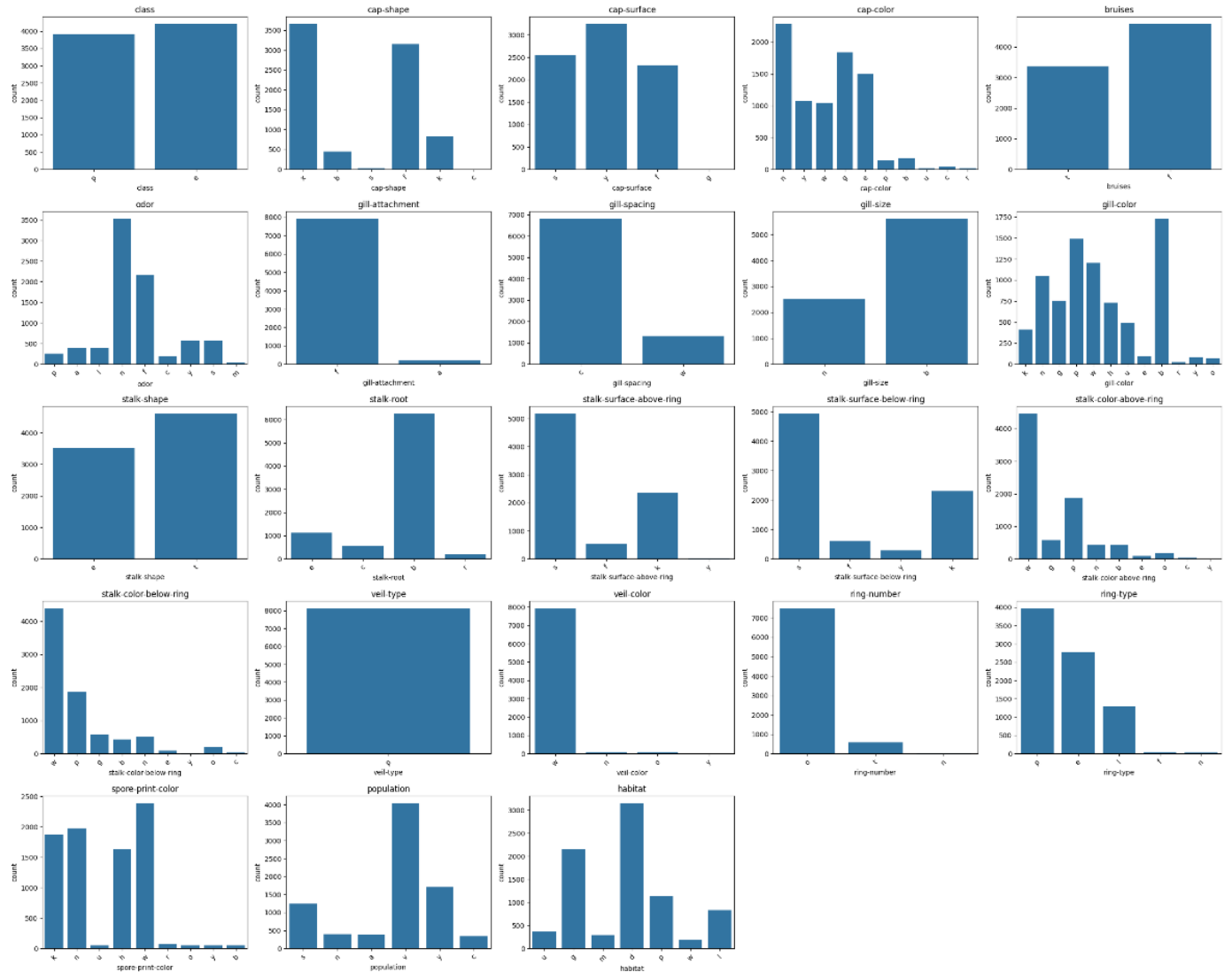


Par exemple, pour la variable **cap-shape** (forme du chapeau), nous avons visualisé le nombre de valeurs pour chaque catégorie

- **x : conique**

Mariam Boughizene Mohsen Allani

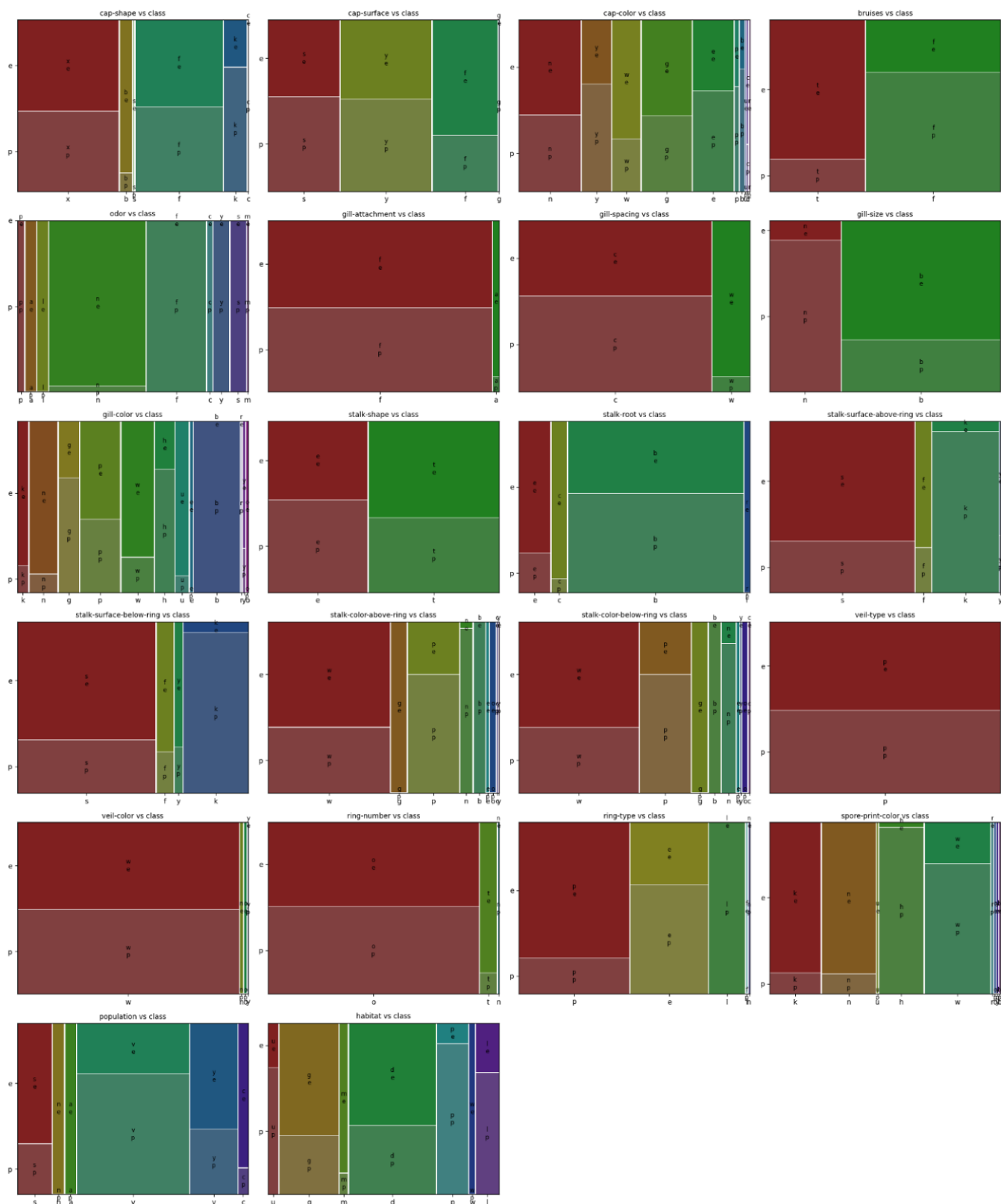
- **b : convexe**
- **f : flared**
- **k : plate**
- **c : clavate**



Nous avons utilisé des graphiques de type mosaïque pour explorer la relation entre chaque caractéristique et la classe des champignons (comestible ou toxique). Chaque graphique montre la répartition des différentes catégories de la caractéristique par rapport aux deux classes possibles : comestible (e) et toxique (p).

Cela nous a permis d'observer comment les différentes valeurs des caractéristiques sont associées à la classe des champignons

Mariem Boughizene Mohsen Allani



L'axe **horizontal** représente les différentes catégories de la caractéristique

L'axe **vertical** représente les deux classes : **comestible (e)** et **toxique (p)**.

Mariem Boughizene

Mohsen Allani

La taille de chaque bloc dans le graphique correspond au nombre d'occurrences d'une combinaison spécifique entre la caractéristique et la classe.

Par exemple, un grand bloc pour la valeur p de veil-type et la classe **toxique (p)** indique qu'il y a beaucoup de champignons avec cette caractéristique qui sont toxiques.

Chaque valeur de veil-type est répartie presque uniformément entre la classe **comestible** et la classe **toxique**, cela peut signifier que cette caractéristique ne discrimine pas bien les deux classes, et qu'elle n'est probablement pas très utile pour la prédiction.

Si un bloc est **significativement plus grand que l'autre**, cela peut indiquer une forte association entre cette catégorie de la caractéristique et une classe particulière.

Par exemple, si la catégorie bruises (présence de bleuissement) montre un grand bloc pour la classe **comestible** lorsque la valeur est t (présence de bleuissement), et un petit bloc pour la classe **toxique** lorsque la valeur est f (absence de bleuissement), cela suggère que le bleuissement peut être un facteur important pour prédire si un champignon est comestible.

Étape 4 : Encodage des variables catégorielles :

Les données étant principalement composées de variables catégorielles (des lettres représentant des catégories), nous avons converti ces catégories en valeurs numériques. Cela est nécessaire car les algorithmes d'apprentissage automatique comme KNN ou l'arbre de décision ne peuvent pas traiter directement des chaînes de caractères. Pour cela, nous avons utilisé un encodeur (LabelEncoder) pour transformer chaque catégorie en un nombre entier.

Étape 5 : Sélection des variables importantes (test du χ^2) :

Nous avons utilisé le test du χ^2 pour mesurer l'importance de chaque caractéristique par rapport à la variable à prédire (comestible ou toxique). Les caractéristiques avec un score élevé (comme gill-color, ring-type ou gill-size) sont considérées comme plus utiles pour faire de bonnes prédictions.

Chi2 Score élevé = forte dépendance avec class = utile pour le modèle

p-value faible (ex: < 0.05) = résultat statistiquement significatif

Après avoir analysé les scores du test du χ^2 , nous avons supprimé certaines variables (cap-color, veil-color, gill-attachment, veil-type, odor) jugées peu informatives pour améliorer la performance et la simplicité des modèles.

Mariam Boughizene

Mohsen Allani

Étape 7 : Séparation des données :

Nous avons divisé notre ensemble de données en deux parties :

- **Entraînement (80%)** : pour apprendre le modèle,
- **Test (20%)** : pour évaluer ses performances.

Cela permet de tester la capacité du modèle à généraliser sur de nouvelles données.

Étape 8 : KNN :

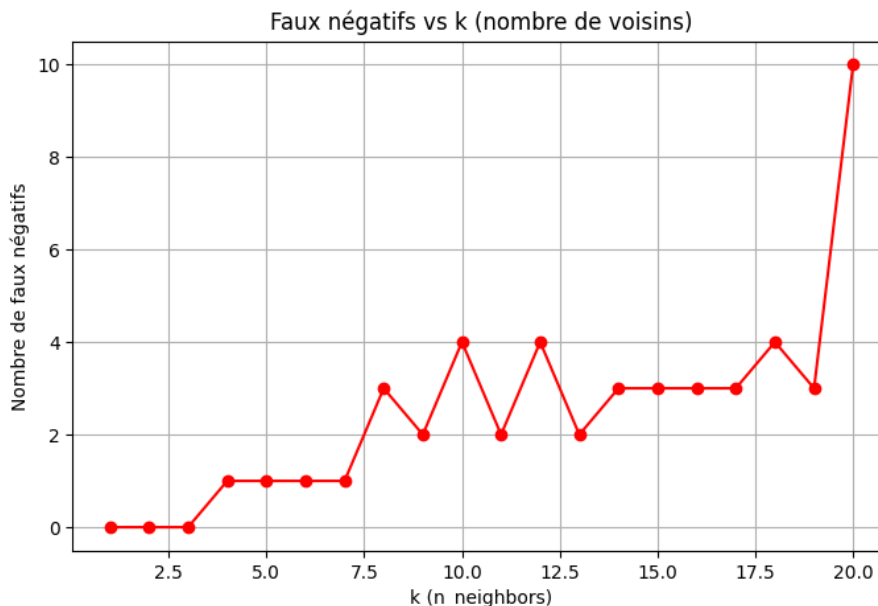
1) Choix du meilleur k pour KNN :

Nous avons testé plusieurs valeurs de k (le nombre de voisins) pour le modèle **K-Nearest Neighbors (KNN)**.

Nous allons choisir le k qui minimise les erreurs où un champignon non comestible est prédit comme comestible, car ce type d'erreur est dangereux.

Deux approches ont été utilisées :

- **Validation simple** : on teste k de 1 à 20



- **Recherche par grille (GridSearchCV)** avec validation croisée pour trouver automatiquement le meilleur k.

Le KNN (K-Nearest Neighbors) se base sur **la distance** entre les exemples pour prédire la classe d'un nouvel échantillon

Mariem Boughizene

Mohsen Allani

Le meilleur score a été obtenu avec **k = 1**, ce qui signifie que la classe d'un champignon est très bien prédite en regardant uniquement le plus proche voisin.

2) Comparaison des métriques de distance pour KNN :

Nous avons comparé différentes **métriques de distance** pour le modèle KNN avec **k = 1**, afin de voir celle qui offre la meilleure performance :

- **Métriques classiques testées :**

- euclidean (euclidienne)
- manhattan
- chebyshev
- Minkowski

- **Métriques alternatives :**

- cosine
- jaccard

- **Résultats :**

Métrique	Moyenne de validation croisée
manhattan	1.0000
minkowski	0.9994
euclidean	0.9994
cosine	0.9992
chebyshev	0.9955
jaccard	0.9798

La **métrique de Manhattan** donne les meilleurs résultats, avec une précision parfaite sur les données d'entraînement (validation croisée 5-fold).

3) Cross Validation :

Mariem Boughizene

Mohsen Allani

La validation croisée est une technique utilisée pour évaluer la performance d'un modèle en le testant sur plusieurs sous-ensembles des données d'entraînement

Les résultats de **la validation croisée** montrent que le modèle a obtenu **une précision parfaite de 1.0** pour chaque partition de données, ce qui signifie que le modèle prédit correctement toutes les instances des jeux de données de validation.

4) Précision d'entraînement (Accuracy) :

Après avoir entraîné le modèle K-Nearest Neighbors (KNN) avec le meilleur paramètre k et la métrique de distance 'manhattan', la précision d'entraînement a été calculée. Le modèle a obtenu une précision de **1.0** (100%) sur l'ensemble d'entraînement, ce qui signifie qu'il a parfaitement classé toutes les instances de cet ensemble.

5) Précision finale sur le jeu de test (Accuracy sur test set) :

Après avoir évalué le modèle KNN sur l'ensemble de test, nous avons obtenu une **précision de 1.0** (100%). Cela signifie que le modèle a parfaitement classé toutes les instances de l'ensemble de test.

Rapport de classification :

Le rapport de classification pour les classes 0 (toxique) et 1 (comestible) présente les résultats suivants :

- **Précision** : 1.00 pour les deux classes, ce qui signifie que le modèle n'a fait aucune erreur de classification pour ces deux catégories.
- **Rappel** : 1.00 pour les deux classes, ce qui indique que toutes les instances de chaque classe ont été correctement identifiées.
- **Score F1** : 1.00 pour les deux classes, ce qui est le meilleur score possible, indiquant un excellent équilibre entre la précision et le rappel.

Le modèle KNN a montré des performances parfaites sur l'ensemble de test, avec des scores parfaits de précision, rappel et F1

Étape 9 : arbre de décision:

1. Pré-élagage :

Avant de former le modèle d'arbre de décision, nous avons effectué une recherche par grille (GridSearchCV) pour identifier les meilleurs hyperparamètres, en particulier pour les critères de pré-élagage. Cette recherche permet de tester plusieurs combinaisons de paramètres pour déterminer celle qui donne la meilleure performance en termes de précision.

Les meilleurs paramètres obtenus pour l'arbre de décision étaient les suivants :

Mariam Boughizene

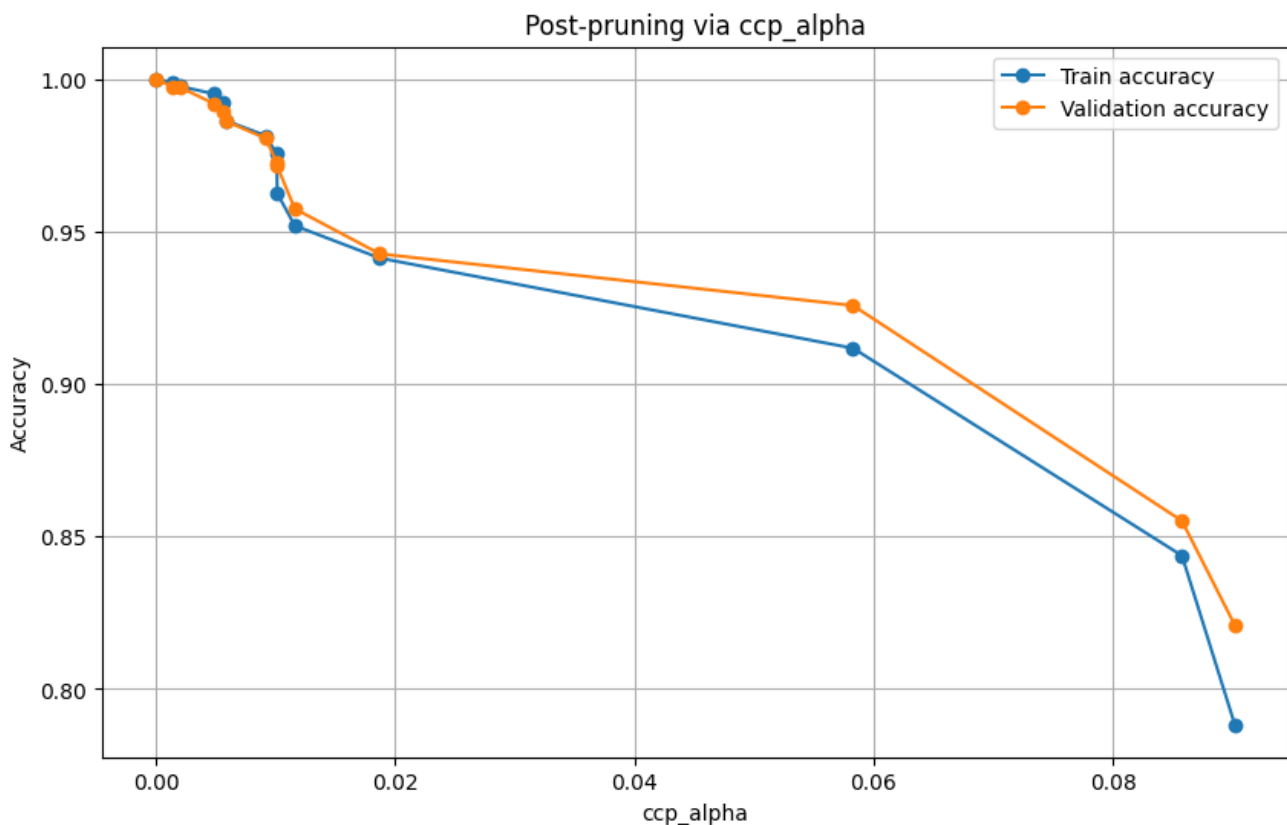
Mohsen Allani

- `max_depth = 10`
- `min_samples_split = 2`
- `min_samples_leaf = 1`

2. Post-élagage :

Après l'entraînement de l'arbre de décision avec les meilleurs paramètres trouvés via la recherche par grille, nous avons effectué un post-pruning (élagage) en utilisant la technique de **coût de complexité de l'élagage (`ccp_alpha`)**. Cette méthode permet de réduire la taille de l'arbre tout en maintenant sa précision, en élaguant les branches de l'arbre qui n'apportent pas suffisamment de gain en précision.

On a créé ce graphe :



Et comme resulta on a trouve Meilleur `ccp_alpha` : 0.0

3. Évaluation :

Après avoir appliqué le post-pruning avec `ccp_alpha` et sélectionné le meilleur modèle, nous avons évalué la performance de l'arbre de décision sur l'ensemble de test.

Le modèle a obtenu une précision parfaite de **1.0** sur l'ensemble de test, ce qui signifie qu'il a correctement classé toutes les instances dans les deux classes.

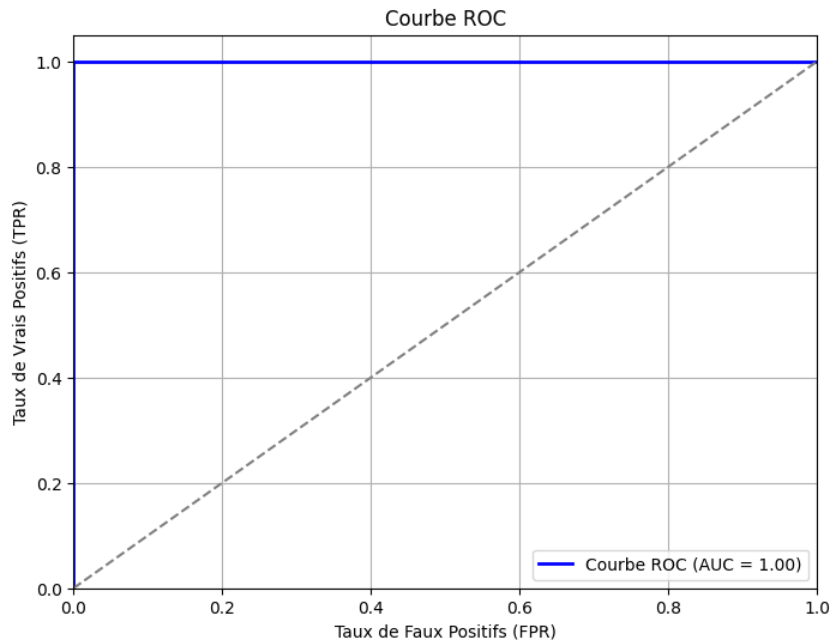
Le rapport de classification montre des résultats excellents pour chaque classe avec une précision, un rappel et un F1-score égaux à **1.00**. Ces valeurs indiquent que le modèle n'a commis aucune erreur lors de la classification.

Mariam Boughizene

Mohsen Allani

4. ROC et AUC :

La performance du modèle a été évaluée avec la courbe **ROC** et l'**AUC** (Area Under the Curve). L'AUC de 1.00 montre que le modèle a une capacité parfaite à discriminer entre les classes. La courbe ROC illustre **le taux de vrais positifs (TPR)** contre **le taux de faux positifs (FPR)**. Un AUC de 1.00 signifie que le modèle est parfait.



Étape 10 : Comparaison :

Les deux modèles (KNN et Decision Tree) ont donné les mêmes résultats parfaits.

- Meme Accuracy finale sur test : 1.0
- Meme Rapport de classification
- Meme Moyenne validation croisée : 1.0

Étape 11 : Prétraitement et pipeline avec KNN :

Un pipeline a été créé pour automatiser le prétraitement des données et l'entraînement du modèle KNN. Le prétraitement inclut :

1. **Suppression de colonnes inutiles.**
2. **Imputation des valeurs manquantes** avec la stratégie `most_frequent` (valeur la plus fréquente).
3. **Encodage des variables catégorielles** avec un **OrdinalEncoder**.

Le modèle **KNN** utilise **1 voisin** et la **métrique de Manhattan**. Le pipeline permet d'effectuer l'entraînement et la prédiction de manière fluide, intégrant toutes les étapes nécessaires.

Mariam Boughizene

Mohsen Allani

Étape 12 : Prétraitement et pipeline avec l'Arbre de Décision :

Un **pipeline** a été conçu pour intégrer le prétraitement des données et l'entraînement d'un modèle **Arbre de Décision** :

1. **Suppression des colonnes** non pertinentes (e.g., cap-color, veil-color).
2. **Imputation des valeurs manquantes** avec la valeur la plus fréquente.
3. **Encodage des variables** catégorielles avec **OrdinalEncoder**.

Le modèle d'**Arbre de Décision** est configuré avec des paramètres optimisés (profondeur maximale = 10, ccp_alpha = 0.0), et une division des données en **train** et **test** (80%/20%).

Ce pipeline permet un traitement complet des données, l'entraînement du modèle et les prédictions sur l'ensemble de test.

Étape 13 : Evaluation après Pipeline :

Les deux modèles, **KNN** et **Arbre de Décision**, ont montré des performances parfaites sur l'ensemble de test, avec une **accuracy** de 1.0 pour les deux.

- **KNN (k-Nearest Neighbors) :**
- **Accuracy** : 1.0
- **Classification Report** : Toutes les métriques (précision, rappel, f1-score) sont égales à 1 pour les deux classes (0 et 1).

Matrice de confusion : Le modèle prédit correctement toutes les instances des deux classes, sans aucune erreur.

Arbre de Décision :

- **Accuracy** : 1.0
- **Classification Report** : De même, toutes les métriques sont égales à 1 pour les deux classes (0 et 1).
- **Matrice de confusion** : Le modèle a également une prédiction parfaite, sans erreurs.

Dans les deux cas, les matrices de confusion montrent que les prédictions sont parfaites, sans faux positifs ni faux négatifs.

Mariam Boughizene
Mohsen Allani

